

Vulnerability Patching via Structure-Aware Retrieval-Augmented Generation

Master's Thesis Progress

Lívia Cavalcanti

June 2026

What Is a Vulnerability?

NIST FIPS 200 [1]

A **vulnerability** is a weakness in an information system, system security procedures, internal controls, or implementation that could be exploited by a threat source.

Example:

Use-After-Free

The program references memory after it has been freed, leading to undefined behavior.

```
struct sock *alloc_socket(  
    struct socket *sock) {  
    struct sock *sk;  
  
    sk = sk_alloc();           // allocate  
  
    sock->sk = sk;             // alias created  
  
    if (!chan_create()) {  
        sk_free(sk);          // free original  
        return NULL;         // BUG: alias dangling!  
    }  
    return sk;  
}
```

Inspired by CVE-2024-56605 (Linux kernel) — a real-world use-after-free vulnerability.

CWE = vulnerability *type* $\longleftrightarrow$ **CVE** = vulnerability *instance*

Tracing the Vulnerability (1/4): Allocation

```
struct sock *alloc_socket(  
    struct socket *sock) {  
    struct sock *sk;  
  
    sk = sk_alloc();           // allocate  
  
    sock->sk = sk;             // alias created  
  
    if (!chan_create()) {  
        sk_free(sk);          // free original  
        return NULL;  
    }  
    return sk;  
}
```

- ✓ **sk is allocated** — a new object is created on the heap

Tracing the Vulnerability (2/4): Alias Creation

```
struct sock *alloc_socket(  
    struct socket *sock) {  
    struct sock *sk;  
  
    sk = sk_alloc();           // allocate  
  
    sock->sk = sk;             // alias created  
  
    if (!chan_create()) {  
        sk_free(sk);          // free original  
        return NULL;  
    }  
    return sk;  
}
```

- ✓ sk is allocated
- ✓ sock->sk **becomes an alias** — two pointers now reference the same object

Tracing the Vulnerability (3/4): Free

```
struct sock *alloc_socket(  
    struct socket *sock) {  
    struct sock *sk;  
  
    sk = sk_alloc();           // allocate  
  
    sock->sk = sk;             // alias created  
  
    if (!chan_create()) {  
        sk_free(sk);          // free original  
        return NULL;  
    }  
    return sk;  
}
```

- ✓ sk is allocated
- ✓ sock->sk becomes an alias
- ✓ **On error, sk is freed** — the object is deallocated

Tracing the Vulnerability (4/4): Dangling Pointer

```
struct sock *alloc_socket(  
    struct socket *sock) {  
    struct sock *sk;  
  
    sk = sk_alloc();           // allocate  
  
    sock->sk = sk;             // alias created  
  
    if (!chan_create()) {  
        sk_free(sk);          // free original  
        return NULL;          // BUG: alias dangling!  
    }  
    return sk;  
}
```

- ✓ sk is allocated
- ✓ sock->sk becomes an alias
- ✓ On error, sk is freed
- ✗ sock->sk still points to freed memory!

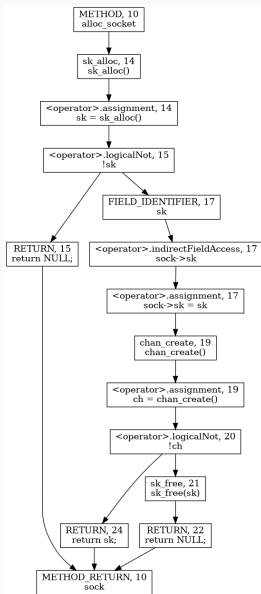
Pattern:

Free without nullifying all aliases

⇒ **Use-After-Free**

This pattern leaves a structural fingerprint in the code graph.

Code Property Graph: Full Joern [2] Output

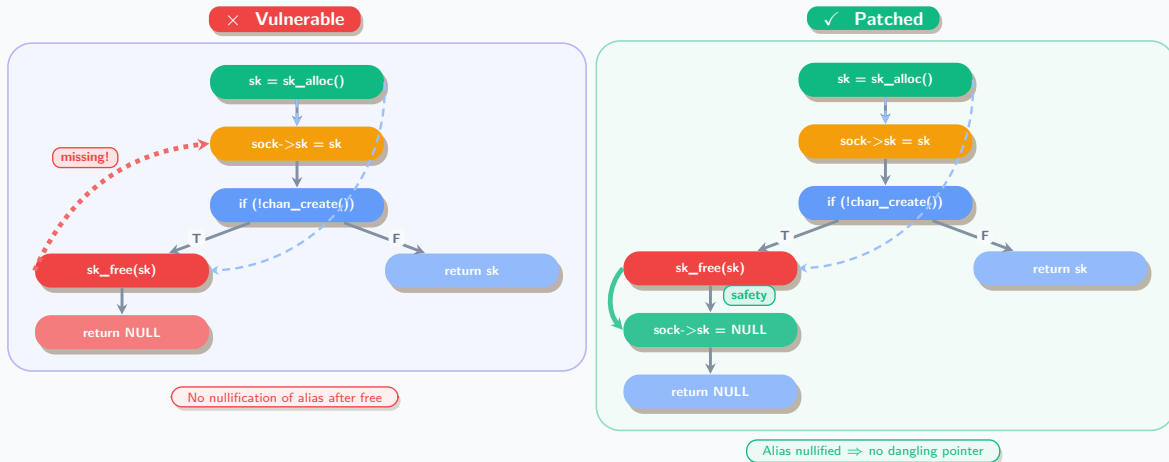


What the CPG encodes:

- **AST** — syntax structure
- **CFG** — execution order
- **DDG** — data flow (def $\rightarrow$ use)
- **CDG** — control dependencies

Full graph is detailed — let's simplify to the vulnerability-relevant subgraph.

Structure Reveals the Vulnerability



Takeaway: The structural difference between vulnerable and patched code is a single edge/node — vulnerabilities leave fingerprints in the graph.

Motivation: Why Current Approaches Fall Short

The Gap

Industrial compliance standards demand *auditable*, automated remediation [3]

Existing Approaches Fall Short

- Graph classifiers: *no knowledge reuse*
- Agentic approaches: *ignore structure*
- LLM failures:
 - **84–94%** of AI patches are still bypassable [4]
 - **91.6%** failure in vulnerability identification [5]

Hallucinated fixes $\Rightarrow$ false sense of security [6]

Existing Approach Strengths

- Static analysis for vulnerability detection is well-established [2]
- Agentic RAG approaches show promising results in code repair [7, 8]

Our Approach

RAG + Static Analysis: retrieve known structural patterns

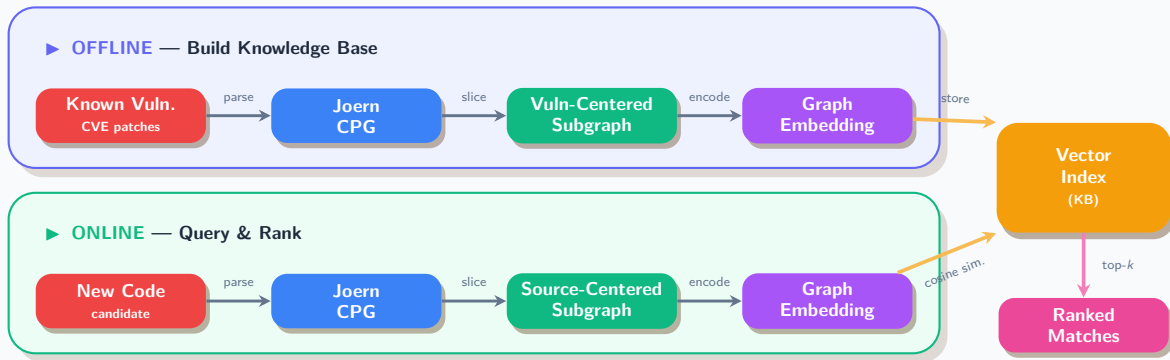
Agentic generation: produce fixes that are *correct **and** explainable*

Research Questions

RQ1: How effective are graph-based embeddings at retrieving relevant vulnerability fix patterns from the knowledge base?

RQ2: What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

System Architecture: Graph-RAG Pipeline



Key idea: Identical pipeline for both tracks — only the *input* differs. **Offline** indexes *known* vulnerability patterns; **Online** queries *unseen* code against them.

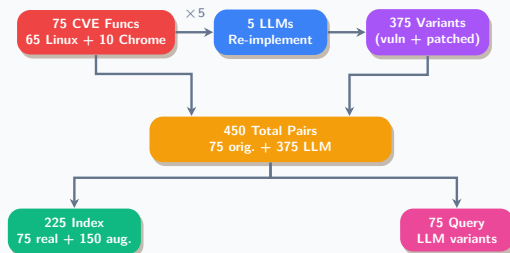
Feasibility Study: AutoPatch Dataset (n=225)

Goal: Validate the concept using a controlled dataset and define baselines.

AutoPatch Dataset:

- 75 CVE functions (65 Linux Kernel, 10 Chromium)
- 5 LLMs re-implement each $\rightarrow$ 375 variant pairs

Can structural graph embeddings capture vulnerability patterns as effectively as pretrained CodeBERT?



Evaluation Metrics

Hit@1

Success rate at rank 1

$$\frac{1}{|Q|} \sum_q \mathbb{1}[\text{rank}_q = 1]$$

Fraction of queries where the correct match is ranked **first**.

Hit@5

Success rate in top 5

$$\frac{1}{|Q|} \sum_q \mathbb{1}[\text{rank}_q \leq 5]$$

Fraction of queries where the correct match appears in **top 5**.

MRR

Mean Reciprocal Rank

$$\frac{1}{|Q|} \sum_q \frac{1}{\text{rank}_q}$$

Average inverse rank across queries — rewards **higher** placements.

CWE Recall

Type coverage

$$\frac{|\text{CWE}_{\text{retrieved}}|}{|\text{CWE}_{\text{index}}|}$$

Fraction of distinct CWE types correctly retrieved **at least once**.

Embedding Pipeline: From Graph to Vector

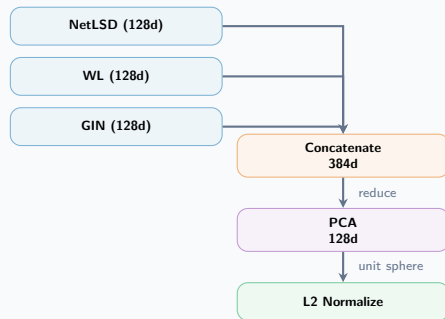
Structure-Only Embedders (no training required):

Embedder	Dim	Mechanism
NetLSD [9]	128	Heat kernel trace on graph Laplacian spectrum (100 timescales 10^{-2} – 10^2)
WL [10]	128	Weisfeiler-Lehman color refinement (4 iters) → sum pool → linear proj.
GIN [11]	128	3-layer Graph Isomorphism Network (random weights, frozen) → mean+sum readout

Semantic Baseline:

Embedder	Dim	Mechanism
CodeBERT [12]	768	Pretrained transformer; CLS token on linearized code

Combined Pipeline:



PCA: fit on index set, retains >95% variance.
All final vectors: **128d**, L2-normalized.

Node features: 11-class CPG node type (one-hot) — METHOD, CALL, IDENTIFIER, LITERAL, RETURN, CONTROL_STRUCTURE, ...

Phase 1: Feasibility — AutoPatch Dataset (n=225)

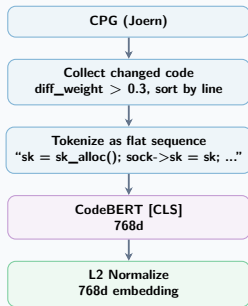
Embedder	H@1	H@5	MRR	CWE R.
<i>Structure-Only</i>				
Combined	0.8904	0.9589	0.9162	0.4978
GIN	0.7260	0.8767	0.7879	0.4427
WL	0.6986	0.7945	0.7363	0.4386
<i>CodeBERT pretrained</i>				
CodeBERT (baseline)	0.1533	0.2200	0.1826	0.2933
CodeBERT (seq)	0.7534	0.8630	0.7927	0.4121
CodeBERT (pat)	0.7123	0.8356	0.7677	0.4147

Source: `experiments/output/checkpoint5/20260505_141347_experiment_f59ab1/results.json`

RQ1: validated — structural *graph* representations are *effective* for vulnerability pattern retrieval in a synthetic dataset.

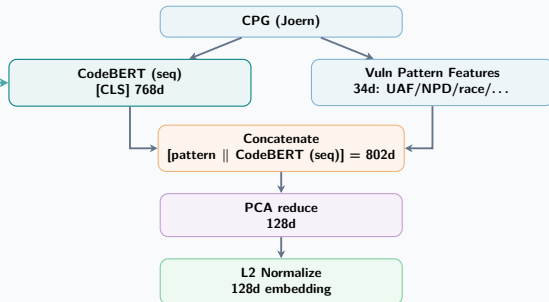
CodeBERT Embeddings — Sequence vs. Pattern

CodeBERT (seq)



Text-only: structure is **discarded**.

CodeBERT (pat)



Fuses **graph structure** with semantics.

Phase 2: Generalization — CVEfixes Dataset

Question: Does the approach generalize to real-world vulnerabilities?

Real-World Dataset

CVEFixes [13] datasets is large dataset with real-world CVEs collected from the U.S. National Vulnerability Database, patches, and code changes.

Metric	Value
CVEs	11,873
Fix commits	12,107
File changes	51,342
Method changes	277,948
Repositories	4,249

Sampling criteria

Structurally distinguishable CWEs (e.g. UAF vs. OOB) to test pattern matching, not just code similarity
Balanced superclass(CWE) up to 20 entries and subclasses (CVE) to ensure retrieval feasibility

Parameter	Value
Total pairs	136
Index set	109
Query set	27
Unique CVEs (index)	60
Unique CVEs (query)	25
CWE classes	7

Phase 2: Pipeline Verification Results

Same-CWE Retrieval (n=109 index, 27 query)

Embedder	Hit@1	Hit@5	MRR	CWE Recall
GIN	0.259	0.778	0.315	0.202
Combined	0.111	0.778	0.223	0.189
CodeBERT (baseline)	0.080	0.1518	0.098	0.141
CodeBERT (seq)	0.556	0.926	0.555	0.264
CodeBERT (pat)	0.519	0.889	0.549	0.276

Source: cvefixes_experiments/output/pipeline_verification/results.json

RQ1 validated on real-world dataset with same-CWE retrieval up to 93% at $k=5$.

Per-CWE Breakdown (CodeBERT Pattern, CWE Hit@k)

CWE	N	Hit@1	Hit@5	Hit@10
Integer Overflow (190)	4	75%	100%	100%
Input Validation (20)	4	75%	75%	100%
Resource Consump. (400)	3	67%	100%	100%
Use After Free (416)	4	50%	100%	100%
NULL Ptr Deref (476)	5	40%	80%	80%
Race Condition (362)	3	33%	100%	100%
OOB Write (787)	4	25%	75%	75%
Overall	27	51.9%	88.9%	92.6%

Source: same file (CodeBERT Pattern cell)

Highlights areas for improvement in embedding design or slicing strategy.

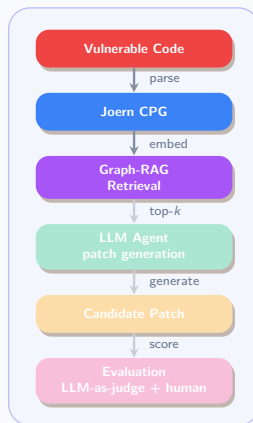
Next Steps: Research Question 2

RQ1: How effective are graph-based embeddings at retrieving relevant vulnerability fix patterns from the knowledge base?

- **Investigate:** Why does combined embedder perform worse than CodeBERT(seq) on CVEFixes?

RQ2: What are the trade-offs of using Graph-RAG in the vulnerability patching pipeline?

1. Integrate the retrieval layer with patch agent
2. Run **comparative experiments** against baseline approaches:
 - zero-shot open-source models
 - AutoPatch
3. **Validate** the end-to-end patching **pipeline** with industry professionals.



Questions?

Livia Cavalcanti |  graph-rag/

Appendix: CPG Deep Dive

The Code Property Graph encodes four types of relationships:

AST: Abstract syntax tree (structure of code)

CFG: Control flow graph (which statements execute in sequence)

DDG: Data dependency graph (variable definitions and uses)

CDG: Control dependency graph (which statements depend on conditionals)

Joern exposes all four via traversals; KB retrieval compares the combined structure to find similar vulnerabilities.

Appendix: Joern Query - Finding the Pattern

Goal: Find `free()` calls with dangling aliases.

```
val uafCandidates = cpg.call.name(".*free.*")
  .map { freeCall =>
    val freedVar = freeCall.argument(1).code
    val aliases = freeCall.method.assignment
      .where(_ .source.isIdentifier.name(freedVar))
      .target.code.l
    val freeLineNo = freeCall.lineNumber.get
    val nullified = freeCall.method.assignment
      .where(_ .lineNumber.map(_ > freeLineNo))
      .target.code.l
    val dangling = aliases.filterNot(nullified.contains)
      (freeCall, dangling)
  }
```

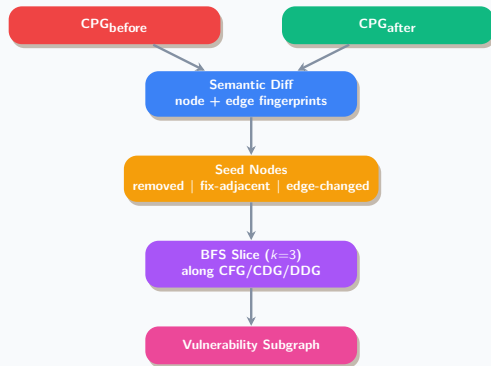
Output: [UAF] `sk_free(sk)` - dangling: `sock->sk`

Appendix: Vulnerability-Aware Code Slicing

Goal: Extract a minimal subgraph capturing the vulnerability context from the full CPG.

Algorithm (3 phases):

1. **Semantic Node Diff** — Match nodes by fingerprint (labelV, CODE, LINE_NUMBER) between pre-/post-patch CPGs. Identify removed, added, and fix-adjacent nodes.
2. **Semantic Edge Diff** — Detect structural changes (rewired control/data flow).
3. **Bounded Program Slice** — BFS $k=3$ hops along flow edges (CFG, CDG, REACHING_DEF, PDG, DDG) from seed nodes.



Diff Label	Weight
removed	1.0
fix_adjacent	0.8
edge_changed	0.6
context	0.2

Appendix: CodeBERT Pattern Embedding — Detail

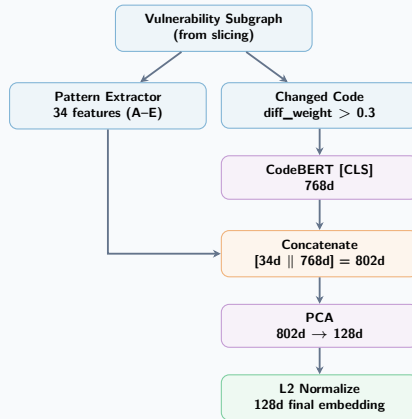
34d Feature Vector — 5 Groups:

All features require CPG edge connectivity (REACHING_DEF, CDG, CFG).

Grp	Dim	Features & Mechanism
A: Vuln Flow (8)	A1	UAF: free $\xrightarrow{\text{REACH_DEF}}$ identifier (2-hop)
	A2	NPD: ptr deref w/o null-check in CDG preds
	A3	Unchecked return: CALL $\nrightarrow$ CONTROL_STRUCT
	A4	Lock/unlock imbalance: $ L - U / (L + U + 1)$
	A5	Unguarded arithmetic: no bounds in CDG neighbourhood
	A6	Use w/o def: IDENTIFIER w/o incoming REACH_DEF
	A7	Alloc/free imbalance: $ A - F / (A + F + 1)$
	A8	Cast in data flow: cast $\xleftarrow{\text{REACH_DEF}}$ changed
B (8)	B1-8	Edge type distribution among changed nodes AST, CFG, CDG, REACH_DEF, REF, ARG, RECV, CALL
C (6)	C1-6	Boundary flow: changed $\rightarrow$ ctx vs ctx $\rightarrow$ changed per CFG, CDG, REACHING_DEF (2 dirs $\times$ 3 types)
D (6)	D1-6	Diff topology: changed frac, density, #components, mean/max degree, internal vs crossing edge ratio
E (6)	E1-6	Node role distribution of changed nodes CALL, CTRL_STRUCT, ID, LITERAL, BLOCK, OTHER

Node classification uses regex on CODE attr (e.g. `free|kfree|vfree` for free nodes). Flow features traverse CPG edges.

Full Pipeline:



Appendix: Joern Generated graph

TODO: add graphml example

References I

- [1] National Institute of Standards and Technology. *Minimum Security Requirements for Federal Information and Information Systems*. Federal Information Processing Standards Publication FIPS PUB 200. NIST, Mar. 2006. URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.200.pdf>.
- [2] Joern Project. *Joern – The Bug Hunter’s Workbench*. 2024. URL: <https://joern.io>.
- [3] International Organization for Standardization. *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. Standard. Geneva, Switzerland: International Organization for Standardization, 2022. URL: <https://www.iso.org/standard/27001>.
- [4] Adiba Mahmud, Yasmeen Rawajfih, and Fan Wu. “A Systematic Evaluation of AI-Generated Security Patches: Analyzing and Exploiting AI-Generated Fixes”. In: *2025 IEEE International Conference on Artificial Intelligence Testing (AITest)*. 2025, pp. 224–231. DOI: 10.1109/AITest66680.2025.00035.

References II

- [5] Kylie McClanahan et al. “When ChatGPT Meets Vulnerability Management: The Good, the Bad, and the Ugly”. In: *2024 International Conference on Computing, Networking and Communications (ICNC)*. 2024, pp. 664–670. DOI: 10.1109/ICNC59896.2024.10555953.
- [6] Tadesse ZeMicheal et al. “LLM agents for vulnerability identification and verification of CVEs”. In: *CAMLIS'24: Conference on Applied Machine Learning for Information Security*. 2024.
- [7] Minjae Seo et al. “AutoPatch: Multi-Agent Framework for Patching Real-World CVE Vulnerabilities”. In: *ArXiv abs/2505.04195 (2025)*. URL: <https://api.semanticscholar.org/CorpusID:278368151>.
- [8] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. “Repairagent: An autonomous, llm-based agent for program repair”. In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE. 2025, pp. 2188–2200.

References III

- [9] Anton Tsitsulin et al. “NetLSD: Hearing the Shape of a Graph”. In: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '18. London, United Kingdom: Association for Computing Machinery, 2018, pp. 2347–2356. ISBN: 9781450355520. DOI: 10.1145/3219819.3219991. URL: <https://doi.org/10.1145/3219819.3219991>.
- [10] Nino Shervashidze et al. “Weisfeiler-Lehman Graph Kernels”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2539–2561.
- [11] Keyulu Xu et al. “How Powerful are Graph Neural Networks?” In: *International Conference on Learning Representations (ICLR)*. 2019. arXiv: 1810.00826.
- [12] Zhangyin Feng et al. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020.
- [13] Guru Prasad Bhandari, Amara Naseer, and Leon Moonen. “CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software”. In: *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)* (2021).